

C++ 联合体 `union` 超通俗讲解（只弄懂**内存共用**核心特性）

一、什么是联合体 union

`union` 叫**联合体 / 共用体**，和结构体 `struct` 语法很像，但**内存布局完全不同**。

核心一句话：

联合体里所有成员，共享同一块内存空间；同一时间只能有效保存其中一个成员的数据。

二、union 定义语法

和结构体几乎一样，只是关键字不同：

代码块

```
1  union 联合体名
2  {
3      类型1 成员1;
4      类型2 成员2;
5      类型3 成员3;
6      // ...
7  };
```

示例：

代码块

```
1  union Data
2  {
3      int a;
4      char b;
5      double c;
6  };
```

三、最核心：内存共用特性（重点）

1. 内存大小规则

- **结构体 struct**: 总大小 = 所有成员字节之和 (内存各自独立, 互不占用)
- **联合体 union**: 总大小 = 占用字节最大的那个成员

上面例子:

int(4字节)、char(1字节)、double(8字节)

联合体整体大小 = 最大的 double = 8 字节

2. 所有成员共用同一块起始内存

union 里 a、b、c 起始地址完全一样, 挤在同一块内存里。

3. 赋值一个, 覆盖其他

因为共用内存:

- 给 a 赋值 → b、c 的数据被覆盖失效
- 给 c 赋值 → a、b 全部被覆盖

四、代码直观演示内存共用

代码块

```
1  #include <iostream>
2  using namespace std;
3
4  union Test
5  {
6      int x;
7      short y;
8  };
9
10 int main()
11 {
12     Test t;
13
14     // 给 x 赋值
15     t.x = 100;
16     cout << "x = " << t.x << endl;
17     cout << "y = " << t.y << endl; // y 被覆盖, 值乱掉
18
19     // 给 y 赋值, x 被覆盖
20     t.y = 5;
21     cout << "x = " << t.x << endl; // x 乱掉
22     cout << "y = " << t.y << endl;
```

```
23
24     return 0;
25 }
```

现象：改一个成员，另一个成员直接乱码，就是因为**共用同一块内存互相覆盖**。

五、struct 和 union 最简对比

类型	内存特点	同一时刻	总内存大小
struct 结构体	成员 各占各的内存	所有成员都能同时存数据	所有成员总和（对齐后）
union 联合体	成员 共享同一块内存	只能用一个成员	最大成员的大小

六、union 简单使用场景（了解即可）

- 1. **节省内存：**多个变量不同时使用，共用一块内存省空间；
- 2. **数据类型转换：**用不同成员解读同一段二进制内存（底层硬件、网络编程常用）；
- 3. 嵌入式开发、底层驱动常用，普通业务代码很少用。

七、必背核心总结

- 1. `union` 所有成员**共享同一块内存**；
- 2. 联合体大小 = **最大成员的字节数**；
- 3. 同一时间**只有一个成员有效**，赋值会互相覆盖；
- 4. 和 struct 语法相似，**内存布局完全相反**。

只掌握这些内存共用特性就足够应付考试、基础编程了。

（注：文档部分内容可能由 AI 生成）